

Ejercicio: Gestor de Memoria Compartida Flexible con Procesos Concurrentes

Objetivo:

Implementar un programa en C que simule un gestor de memoria compartida (SHM) capaz de asignar y manejar bloques de datos de tamaño variable para distintos procesos hijos, de forma similar a un pequeño "malloc" cooperativo.

Descripción:

Se debe crear un segmento de memoria compartida que contendrá: - Una estructura de control `SharedMem`, con información sobre bloqueos, offsets, y vectores asignados. - Un área flexible (`unsigned char memory[]`) utilizada para almacenar los datos de cada proceso hijo.

El programa debe: 1. Crear un segmento de memoria compartida de tamaño especificado por el usuario. 2. Lanzar varios procesos hijos, cada uno encargado de leer un archivo de entrada (`data_X.in`) que contiene una cantidad de enteros. 3. Cada hijo debe solicitar un bloque de memoria compartida para almacenar sus datos mediante una función `shm_malloc()`. 4. Si no hay memoria suficiente, el hijo debe imprimir un mensaje de error y finalizar. 5. Al finalizar todos los hijos, el proceso padre debe: - Esperar su finalización. - Mostrar la suma de los datos cargados por cada hijo. - Presentar un resumen de las asignaciones de memoria (PID, offset, longitud, uso total).

Estructuras sugeridas: - `SharedMem`: Contiene la información global del segmento (bloqueo, offset, número de vectores, etc.). - `VecInfo`: Contiene la información individual de cada bloque asignado (PID, offset, longitud).

Funciones requeridas: - `createSHMv()`: Crea el segmento de memoria compartida y devuelve su ID y puntero. - `shm_malloc()`: Asigna memoria dentro del segmento compartido. - `shm_free()`: Libera (o simula liberar) memoria compartida.

Entradas:

```
./programa <tamaño_memoria> <n_hijos>
```

Cada archivo `data_i.in` debe contener:

```
<N>
<valores_enteros>
```

Salida esperada: - Suma de datos procesados por cada hijo. - Resumen de asignaciones. - Información sobre el uso de memoria compartida total.

Desafío adicional: Implementar una variable global `TOTAL_SIZE` que almacene el tamaño total del segmento de memoria y permita verificar si las asignaciones exceden la capacidad disponible.